

Pendências da API AgendaAi — Backend

Data: 18/06/2026

Ambiente validado: <https://agendaaai.bixs.com.br>

Código analisado: api/ (15 controllers, Program.cs, Models/DbAgendaAi.cs)

Total de pendências: 16 (3 exigem deploy de código já corrigido localmente)

P0 — Bloqueia integração

1. GET/PUT /api/ConfigComercio — IDs zerados e 2º save quebra

Endpoints: GET /api/ConfigComercio/{id}, PUT /api/ConfigComercio/Editar-Atendimento/{id}

Problema: O GET em produção retorna idHorarioAtendimento: 0 e idConfigComercio: 0 mesmo com config existente. O PUT busca por esses IDs e retorna 404.

Evidência produção:

```
{ "idHorarioAtendimento": 0, "idConfigComercio": 0, "configuracao":  
{ "comercioId": 1 } }
```

PUT .../Editar-Atendimento/1 → 404 "Horário de atendimento não encontrado."

Código local: Correção já existe em ConfigComercioController.cs (linhas 52–53 no GET; 228–251 fallback no PUT). **Falta deploy.**

Ação: Publicar versão atual do controller.

2. POST /api/ConfigComercio — lógica invertida de ComercioId

Arquivo: ConfigComercioController.cs linhas 122–131

Problema: O if/else está invertido:

- Quando ComercioId == null → busca comércio pelo ID nulo (sempre falha).
- Quando ComercioId está preenchido → ignora o body e resolve comércio só pelo vínculo admin logado.

Ação: Inverter a condição: com ComercioId preenchido, buscar por c.Id == comercioConfiguracao.Configuracao.ComercioId; sem ID, usar vínculo do usuário autenticado.

3. PUT /api/Usuario/Config-Usuario/{id} — parâmetro int incompatível com GUID

Arquivo: UsuarioController.cs linha 138

Problema: O GET já usa string id (GUID), mas o PUT declara ConfigUsuario(int id, ...). GUID na URL → **400**:

```
{ "errors": { "id": ["The value '5de913ee-...' is not valid."] } }
```

Ação: Alterar para string id, buscar com UsuarioId == id (espelhar o GET). Criar registro padrão se não existir (opcional).

P1 — Funcionalidade comprometida

4. POST /api/Login/refresh-token — dependência exclusiva de cookie

Arquivo: LoginController.cs linhas 619–674

Problema:

- Lê refresh token **apenas** de Request.Cookies["refresh_token"] → **401** sem cookie.
- Resposta só retorna { message: "Token renovado!" } — não devolve o novo JWT no body.
- Clientes SPA/mobile que usam só Bearer não conseguem renovar sessão.

Ação: Aceitar { "refreshToken": "..." } no body (ou header dedicado). Retornar { token, expiracao, permissao } no body. Manter cookies como opção para browsers.

5. Typos na role Profissional no login e refresh

Arquivo: LoginController.cs

Linha	Tipo usado
193, 347, 539	"Profffisional"
648	"Profisional"

Problema: Usuários profissionais recebem permissao: "Cliente" no JWT. Ex.: profissional@agendai.dev loga como Cliente.

Ação: Unificar todas as ocorrências para "Profissional". Revisar contas já cadastradas com role errada.

6. POST /api/Agenda/Comercio-Agendar — cliente gravado errado

Arquivo: AgendaController.cs linha 552

Problema: `UsuarioId = agendamento.IdProssional` — o agendamento fica vinculado ao profissional em vez do cliente.

Ação: Usar `agendamento.IdUsuario` (com fallback documentado no contrato).

7. WhatsApp — integração inoperante

Arquivo: WhatsAppController.cs linha 23

Problema:

- `new HttpClient()` no construtor (token desatualizado, risco de socket exhaustion).
- `GET /api/WhatsApp/Obter-QrCode/{id}` → **400** `{"message": "Falha ao obter QR Code"}`.
- Falha TLS/handshake com `api.bixs.com.br`.

Ação: Usar `IHttpClientFactory`. Renovar certificado ou política TLS de `api.bixs.com.br`. Atualizar Bearer antes de cada chamada externa.

P2 — Segurança, contrato e QA

8. GET /api/Pagamentos/Confirmar/{id} — sem autorização de Admin

Arquivo: PagamentosController.cs linhas 103–116

Problema: `[Authorize]` genérico — qualquer autenticado confirma qualquer pagamento pelo ID.

Ação: `[Authorize(Roles = "Admin")]` + validar que o pagamento pertence ao comércio do usuário logado.

9. POST /api/Pagamentos — cálculo incorreto de valor

Arquivo: PagamentosController.cs linhas 146–148

Problema: Se `Valor != 0` e diferente do preço do serviço, o código **soma** o preço ao valor enviado em vez de usar um ou outro.

Ação: Definir regra clara (usar preço do serviço ou valor enviado, nunca somar).

10. GET /api/Agenda/Comercio-Historico/{id} — contrato e filtro quebrados

Arquivo: AgendaController.cs linhas 431–434, 467

Problemas:

- Filtro `periodo` tem bloco vazio — parâmetro ignorado.
- Lista vazia retorna string "Histórico Vazio" em vez de array/objeto JSON (inconsistente com `Comercio/{id}`).

Ação: Implementar filtro `periodo`. Retornar `{ "agendamentos": [] }` ou `[]` quando vazio.

11. DELETE /api/AdminTeste/Deletar-Dados — apaga roles do Identity

Arquivo: DbAgendaAi.cs linhas 14–57, chamado em AdminTesteController.cs linha 53

Problema: `LimparBancoDeDadosManual()` executa `DELETE FROM` em **todas** as tabelas, inclusive `AspNetRoles` e `AspNetUserRoles`. Após reset, `Registrar` com `tipoPermissao Admin/Profissional` pode dar **500** (`NullReferenceException` em `Roles.FirstOrDefault(...).Id`).

Mitigação parcial: `Program.cs` re-seeda roles na inicialização da app — só funciona após restart do serviço.

Ação: Excluir tabelas Identity (`AspNetRoles`, `AspNetUserRoles`, etc.) do script de limpeza, ou re-seedar roles imediatamente após o `DELETE`.

12. POST /api/Login/Registrar — sem null-check em roles

Arquivo: LoginController.cs linhas 441, 445

Problema: `_context.Roles.FirstOrDefault(r => r.Name == "Admin").Id` sem verificar null — **500** se roles foram apagadas e a app não reiniciou.

Ação: Null-check ou garantir roles antes de atribuir (alinhado com correção do item 11).

13. DELETE /api/AdminTeste/Empresas — método incompleto

Arquivo: AdminTesteController.cs linhas 36–44

Problema: Endpoint declarado como `DELETE` mas apenas lista comércios e retorna `Ok(comercios)` — **não deleta nada**.

Ação: Implementar a exclusão ou `remove/renomear` o endpoint para evitar uso incorreto em QA.

14. GET /api/ConfigComercio/{id} sem config — retorno plain text em produção

Arquivo: ConfigComercioController.cs linhas 44–48

Problema: Código local já retorna { "message": "... " }, mas em produção versões antigas devolviam string plain text — quebra parse JSON no frontend.

Ação: Deploy da versão local (mesmo deploy do item 1).

P3 — Infraestrutura e configuração

15. CORS — domínio da API ausente

Arquivo: Program.cs linhas 58–62

Problema: WithOrigins não inclui https://agendaai.bixs.com.br. Chamadas browser diretas à API podem falhar.

Ação: Adicionar origem de produção à policy CORS.

16. IIS 404.0 — Handler StaticFile (intermitente)

Sintoma: Endpoints retornam HTML do IIS em vez de JSON (Handler: StaticFile, pasta física api/ no vhost).

Ação (ops): Verificar web.config / ASP.NET Core Module. Remover pasta física api/ no vhost se interceptar rotas. Republicar e reiniciar app pool.

Deploy pendente (código local já corrigido)

Publicar estes trechos resolve os itens 1 e 14 sem alteração adicional de código:

Arquivo	Linhas	Correção
ConfigComercioController.cs	52–53	Preenche IdHorarioAtendimento e IdConfigComercio no GET
ConfigComercioController.cs	228–251	PUT com fallback por comercioId quando IDs = 0
ConfigComercioController.cs	44–48	Sem config → objeto JSON { message }

Relatório para equipe backend — 18/06/2026. Auditoria: HTTP em produção + leitura estática de api/.